

JAVA SPEZIAL

Arrays, ArrayList, Collections

Alles, was Sie für die Prüfung wissen müssen

Vergleich, Konvertierung, statische Methoden,
Comparable/Comparator, funktionale Interfaces, Lambdas

In diesem Skript:

- Arrays vs. ArrayList – Stärken, Schwächen, Einsatzgebiete
- Konvertierung Arrays ↔ List
- Wichtige statische Methoden aus Arrays und Collections
- Suchen mit `binarySearch`, lineare Suche
- Natürliche Ordnung mit `Comparable`, flexible Ordnung mit `Comparator`
- Funktionale Interfaces, Lambdas

Java: Arrays, ArrayList und Collections

Benjamin Malicke ©

18. März 2026

Inhaltsverzeichnis

1	Einführung und Motivation	3
2	Arrays in Java: Grundlagen, Stärken, Schwächen	3
2.1	Grundidee und Syntax	3
2.2	Stärken von Arrays	3
2.3	Schwächen von Arrays	4
2.4	Arrays im Überblick (Mini-Tabelle)	4
3	ArrayList in Java: Grundlagen, Stärken, Schwächen	5
3.1	Grundidee und Syntax	5
3.2	Stärken von ArrayList	5
3.3	Schwächen von ArrayList	5
3.4	ArrayList im Überblick (Mini-Tabelle)	6
3.5	Wichtige statische Hilfsmethoden für Listen	6
4	Direkter Vergleich: Array vs. ArrayList	7
5	Konvertierung zwischen Array und List	8
5.1	Von Array zu List mit <code>Arrays.asList</code>	8
5.2	Mutable Liste aus einem Array erzeugen	9
5.3	Von List zu Array mit <code>toArray</code>	9
5.4	Stolperfallen bei der Konvertierung	9
5.5	Praxisbeispiel: API erwartet Array	10
6	Statische Methoden von Arrays und Collections	10
6.1	Sortieren von Arrays mit <code>Arrays.sort</code>	10
6.2	Sortieren von Listen mit <code>Collections.sort</code> und <code>List.sort</code>	11
6.3	Minimum und Maximum mit <code>Collections.min</code> und <code>Collections.max</code>	12
7	Suchen mit Arrays und Collections	12
7.1	Lineare Suche: <code>contains</code> und <code>indexOf</code>	13
7.2	Binäre Suche mit <code>Arrays.binarySearch</code>	13
7.3	Binäre Suche mit <code>Collections.binarySearch</code>	14
7.4	Rückgabewert von <code>binarySearch</code> verstehen	14
8	Comparable und Comparator: natürliche und flexible Ordnung	15
8.1	Natürliche Ordnung mit <code>Comparable</code>	15
8.2	Alternative Ordnungen mit <code>Comparator</code>	17
8.3	Hilfsmethoden der Klasse <code>Comparator</code>	17
8.4	Verbindung zu <code>sort</code> , <code>binarySearch</code> , <code>min</code> und <code>max</code>	18

9 Funktionale Interfaces und Lambdas	20
9.1 Von anonymen Klassen zu Lambdas	20
9.2 Was ist ein funktionales Interface?	20
9.3 Lambda-Grundsyntax	21
9.4 Comparator als funktionales Interface	22
9.5 Funktionale Interfaces im Zusammenspiel mit Streams	22
9.6 Zusammenfassung: Warum das für Arrays und ArrayList wichtig ist	23
10 Zusammenfassung und Spickzettel	23
10.1 Arrays vs. ArrayList – Schnellentscheidung	23
10.2 Wichtige Klassen und ihr Zweck	23
10.3 Vergleichen und Sortieren auf einen Blick	24
10.4 Suchen: linear vs. binär	24
10.5 Konvertierung: Arrays $\leftrightarrow$ List	24
10.6 Funktionale Interfaces und Lambdas – Kurzüberblick	25
10.7 Minimal-Spickzettel: „Wenn du X brauchst, nimm Y“	25
A Cheat Sheet: Arrays, ArrayList, Collections, Lambdas	26
B Prüfungs-Checkliste	27

1 Einführung und Motivation

In Java gibt es mehrere Möglichkeiten, mehrere Werte in einer gemeinsamen Struktur zu speichern.

Zwei der wichtigsten Bausteine sind:

- **Arrays**: einfache, sehr effiziente Datenstruktur mit fester Länge
- **ArrayList**: dynamische Liste aus dem *Collections Framework*

Die Wahl zwischen Array und **ArrayList** hat direkte Auswirkungen auf:

- Speicherverbrauch und Performance
- Flexibilität zur Laufzeit (Größenänderung)
- Verfügbarkeit von Komfort-Methoden (Sortieren, Suchen, Filtern)

Dieses Skript zeigt Schritt für Schritt:

1. Grundlagen, Stärken und Schwächen von Arrays und **ArrayList**
2. Direktvergleich und Konvertierung zwischen beiden
3. Wichtige statische Methoden aus `java.util.Arrays` und `java.util.Collections`
4. Vergleichen und Sortieren mit `Comparable` und `Comparator`
5. Einsatz von Lambdas und funktionalen Interfaces

Merkkasten

Merke: feste vs. dynamische Größe

- **Array**: feste Größe, sehr effizient, wenig Komfort-API.
- **ArrayList**: dynamische Größe, komfortable API, etwas mehr Overhead.

2 Arrays in Java: Grundlagen, Stärken, Schwächen

2.1 Grundidee und Syntax

Ein Array speichert eine feste Anzahl von Werten desselben Typs in einem zusammenhängenden Speicherbereich.

```
1  int[] zahlen = new int[5]; // Array mit 5 int-Werten
2  zahlen[0] = 42; // Zugriff über Index
3
4  int[] literal = {1, 2, 3, 4}; // Initialisierung mit Werten
```

Arrays können primitive Typen (`int`, `double`,...) und Referenztypen speichern:

```
1  String[] namen = new String[] { "Anna", "Bernd", "Clara" };
```

2.2 Stärken von Arrays

- Sehr schneller, konstanter Zugriff ($O(1)$) über den Index.
- Unterstützen primitive Typen ohne Autoboxing.
- Geringer Speicher-Overhead, da nur die Daten und die Länge gespeichert werden.

2.3 Schwächen von Arrays

- Größe ist **fest** und muss bei der Erstellung bekannt sein.
- Keine Methoden wie **add**, **remove** oder **contains** direkt am Array.
- Viele Operationen (Sortieren, Suchen, Kopieren) nur über Hilfsklasse `java.util.Arrays`.

2.4 Arrays im Überblick (Mini-Tabelle)

Eigenschaft	Beschreibung
Feste Länge	Größe wird bei der Erstellung festgelegt und kann nicht geändert werden.
Indezzugriff	Direkter Zugriff über ganzzahlige Indizes ab 0.
Datentypen	Primitive und Referenztypen möglich.
Performance	Sehr schnell bei sequentiellm Zugriff und einfachen Schleifen.
API	Komfort-Methoden über <code>java.util.Arrays</code> .
Thread-Sicherheit	Nicht automatisch thread-sicher.
Speicher	Geringer Overhead, gut für große, einfache Datenmen- gen.

Stolperfallen

Typische Stolperfallen bei Arrays

- `ArrayIndexOutOfBoundsException` durch falsche Indizes (z.B. `-1` oder `length`).
- Vergessen, das Array zu initialisieren (`null`-Referenz).
- Versuch, die Größe eines Arrays zur Laufzeit zu ändern, statt ein neues Array anzulegen.

3 ArrayList in Java: Grundlagen, Stärken, Schwächen

3.1 Grundidee und Syntax

`ArrayList` ist eine dynamische Liste aus dem *Java Collections Framework* (Paket `java.util`). Sie basiert intern auf einem Array, kann ihre Größe aber zur Laufzeit flexibel anpassen.

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 List<String> namen = new ArrayList<>();
5 namen.add("Anna");
6 namen.add("Bernd");
7
8 String erster = namen.get(0); // "Anna"
9 int anzahl = namen.size(); // 2
```

Im Gegensatz zu Arrays arbeitet `ArrayList` immer mit **Objekten**. Primitive Typen wie `int` werden automatisch in Wrapper-Klassen (z.B. `Integer`) umgewandelt (Autoboxing).

```
1 ArrayList<Integer> zahlen = new ArrayList<>();
2 zahlen.add(1); // Autoboxing: int -> Integer
3 zahlen.add(2);
```

3.2 Stärken von ArrayList

- **Dynamische Größe:** Elemente können beliebig hinzugefügt oder entfernt werden.
- **Reiche API:** Methoden wie `add`, `remove`, `contains`, `indexOf`, `subList`, ...
- **Gute Integration** mit anderen Collections (`List`, `Set`, `Map`) und mit Streams.
- **Einfache Verwendung:** Code wird oft kürzer und lesbarer als mit rohen Arrays.

3.3 Schwächen von ArrayList

- Arbeitet nur mit Objekten: Primitive Typen verursachen Autoboxing und damit zusätzlichen Overhead.
- Etwas höherer Speicherverbrauch durch Wrapper-Objekte und Verwaltungsstrukturen.
- Zugriff über `get/set` statt direkten Indexzugriff-Syntax wie bei Arrays.
- Wie Arrays nicht automatisch thread-sicher; für parallelen Zugriff sind zusätzliche Maßnahmen nötig.

3.4 ArrayList im Überblick (Mini-Tabelle)

Eigenschaft	Beschreibung
Dynamische Größe	Wächst und schrumpft automatisch bei <code>add</code> und <code>remove</code> .
Datentypen	Nur Objektreferenzen (Wrapper für Primitive wie <code>Integer</code> , <code>Double</code> , ...).
Zugriff	Methoden <code>get(int index)</code> , <code>set(int index, T value)</code> .
API	Umfangreiche Methoden aus dem <code>List</code> -Interface (<code>add</code> , <code>remove</code> , <code>contains</code> , ...).
Integration	Teil des Collections Framework, gut kombinierbar mit <code>Collections.*</code> und Streams.
Thread-Sicherheit	Nicht automatisch thread-sicher (z.B. <code>Collections.synchronizedList</code> nötig).
Performance	Sehr schnell beim sequentiellen Zugriff; Einfügen/Entfernen in der Mitte kann teuer sein (Verschieben von Elementen).

Merkkasten

Merke: Wann ist ArrayList praktisch?

- Wenn die Anzahl der Elemente zur Laufzeit schwankt.
- Wenn Sie häufig Elemente einfügen oder entfernen (vor allem am Ende der Liste).
- Wenn Sie von der umfangreichen Collections-API und Streams profitieren möchten.

Stolperfallen

Typische Stolperfallen bei ArrayList

- Autoboxing-Overhead bei sehr großen Mengen primitiver Werte (z.B. Millionen `int`-Werte).
- Verwechslung von `size()` und physischer Kapazität: `ensureCapacity` ist selten nötig, kann aber bei Performance-Tuning eine Rolle spielen.
- Verwendung von `ArrayList` in Multi-Thread-Umgebungen ohne Synchronisation (Race-Conditions).

3.5 Wichtige statische Hilfsmethoden für Listen

Viele Operationen auf Listen erfolgen nicht direkt über `ArrayList`, sondern über die Utility-Klasse `java.util.Collections`.

```
1  import java.util.*;
2
3  List<Integer> zahlen = new ArrayList<>();
4  zahlen.add(3);
5  zahlen.add(1);
6  zahlen.add(2);
7
8  Collections.sort(zahlen); // sortiert aufsteigend
9  int max = Collections.max(zahlen); // größtes Element
```

Diese statischen Methoden betrachten wir später im Detail (Sortieren, Suchen, `max/min`, `binarySearch`, usw.).

4 Direkter Vergleich: `Array` vs. `ArrayList`

In vielen Situationen kann man sowohl `Arrays` als auch `ArrayList` verwenden. Die folgende Übersicht zeigt die wichtigsten Unterschiede auf einen Blick.

Kriterium	Array	ArrayList
Größe	Feste Länge, wird bei der Erstellung festgelegt und kann nicht geändert werden.	Dynamische Länge, wächst und schrumpft automatisch bei <code>add/remove</code> .
Datentypen	Unterstützt Primitive (<code>int</code> , <code>double</code> ,...) und Referenztypen.	Speichert nur Objektreferenzen (Wrapper für Primitive wie <code>Integer</code> , <code>Double</code> ,...).
Zugriff	Direkter Indexzugriff mit Syntax <code>array[index]</code> .	Zugriff über Methoden <code>get(index)</code> , <code>set(index, value)</code> .
Performance	Sehr effizient, insbesondere für primitive Typen und sequentiellen Zugriff.	Leichter Overhead durch Objekte und Verwaltung, dafür komfortable API.
Speicher	Geringer Overhead, gut für große, einfache Datenmengen.	Mehr Speicherbedarf durch Wrapper-Objekte und interne Struktur.
Flexibilität	Kaum flexibel: Größe fix, keine Methoden wie <code>add/remove</code> .	Sehr flexibel: Elemente können eingefügt, entfernt, gesucht und gefiltert werden.
API-Unterstützung	Viele Operationen über <code>java.util.Arrays</code> (statische Methoden).	Reichhaltige Unterstützung über das <code>List</code> -Interface und <code>java.util.Collections</code> .
Thread-Sicherheit	Nicht automatisch thread-sicher. Synchronisation manuell nötig.	Ebenfalls nicht thread-sicher; kann mit <code>Collections.synchronizedList</code> oder anderen Mechanismen abgesichert werden.
Typische Einsatzzwecke	Numerische Berechnungen, Puffer fester Größe, low-level-nahe Datenstrukturen.	Geschäftslogik, dynamische Listen (z.B. Benutzer, Warenkörbe, GUI-Listen).

Merkkasten

Wann verwende ich was?

- **Array**, wenn:
 - die Größe von Anfang an feststeht,
 - Performance und Speicher eine große Rolle spielen,
 - viele primitive Werte (`int`, `double`,...) verarbeitet werden.
- **ArrayList**, wenn:
 - die Anzahl der Elemente zur Laufzeit schwankt,
 - Sie häufig Elemente hinzufügen/entfernen,
 - Sie von der `Collections`-API und Streams profitieren wollen.

Stolperfallen

Häufige Fehlentscheidungen

- Für große Mengen von `int`/`double` sofort `ArrayList` wählen – der Autoboxing-Overhead kann hier unnötig teuer sein.
- Für dynamische Datenstrukturen (z.B. wachsende Listen) trotzdem Arrays benutzen und ständig kopieren – hier ist `ArrayList` oft deutlich angenehmer.
- API-Sicht vergessen: viele moderne Bibliotheken und Frameworks arbeiten direkt mit `List<T>` statt mit rohen Arrays.

5 Konvertierung zwischen Array und List

Häufig müssen Daten zwischen Arrays und Listen hin- und herkonvertiert werden. Typische Gründe:

- Eine Bibliothek erwartet ein Array, Ihre Anwendung arbeitet aber mit `List<T>`.
- Sie bekommen ein Array (z.B. aus einer älteren API), möchten aber die komfortable `List`-API nutzen.

5.1 Von Array zu List mit `Arrays.asList`

Die Klasse `java.util.Arrays` bietet eine bequeme Methode, um ein Array in eine `List`-Sicht zu verwandeln:

```
1 import java.util.*;
2
3 String[] namenArray = {"Anna", "Bernd", "Clara"};
4 List<String> namenListe = Arrays.asList(namenArray);
5
6 echo = namenListe.get(0); // "Anna"
7 int anzahl = namenListe.size(); // 3
```

Wichtig: die zurückgegebene Liste hat eine **feste Größe**. Die Elemente können geändert werden, aber die Länge der Liste nicht.

```
1 namenListe.set(1, "Berta"); // OK, ändert Element an Index 1
2
3 namenListe.add("Doris"); // !!! UnsupportedOperationException
4 namenListe.remove(0); // !!! UnsupportedOperationException
```

Merkkasten

Merke: `Arrays.asList`

- Gibt eine *fixe* `List`-Sicht auf das ursprüngliche Array zurück.
- Änderungen an der Liste wirken sich auf das zugrunde liegende Array aus und umgekehrt.
- `set(..)` ist erlaubt, `add(..)` und `remove(..)` führen zu `UnsupportedOperationException`.

5.2 Mutable Liste aus einem Array erzeugen

Wenn Sie eine wirklich veränderbare Liste benötigen, erzeugen Sie eine neue `ArrayList` aus der mit `Arrays.asList` erzeugten Sicht:

```
1 String[] namenArray = {"Anna", "Bernd", "Clara"};
2
3 List<String> festeListe = Arrays.asList(namenArray); // feste Größe
4 List<String> mutableListe = new ArrayList<>(festeListe); // wirklich
    dynamisch
5
6 mutableListe.add("Doris"); // OK
7 mutableListe.remove("Bernd");
```

5.3 Von List zu Array mit toArray

Jede `List<T>` kann über die Methode `toArray` in ein Array überführt werden. Der häufigste und empfohlene Weg in modernen Java-Versionen ist:

```
1 List<Integer> zahlenListe = new ArrayList<>();
2 zahlenListe.add(1);
3 zahlenListe.add(2);
4 zahlenListe.add(3);
5
6 Integer[] zahlenArray = zahlenListe.toArray(new Integer[0]);
```

Merkkasten

Merke: toArray

- `T[] arr = liste.toArray(new T[0]);` ist ein verbreitetes Idiom.
- Der Typ des Arrays wird durch das übergebene "Muster-Array" bestimmt.
- Die Größe des Ergebnis-Arrays entspricht automatisch der `size()` der Liste.

5.4 Stolperfallen bei der Konvertierung

Stolperfallen

Typische Fehler bei Arrays ↔ List

- **Vergessen, dass `Arrays.asList` feste Größe hat:** `add` und `remove` schlagen zur Laufzeit fehl.
- **Erwarten, dass Änderungen an der `ArrayList` das ursprüngliche Array verändern:** Nur die von `Arrays.asList` zurückgegebene feste Liste ist direkt mit dem Array gekoppelt. Eine neue `ArrayList<>(...)` ist unabhängig.
- **Rohes `Object[]` statt typisiertem Array:** Der Parameter-lose Aufruf `toArray()` gibt ein `Object[]` zurück, das oft noch gecastet werden müsste. Besser gleich `toArray(new T[0])` nutzen.

5.5 Praxisbeispiel: API erwartet Array

```
1 // Beispiel: Eine (fiktive) Bibliotheksmethode erwartet ein String-Array
2 public void sendeNamen(String[] namen) {
3     //...
4 }
5
6 List<String> namenListe = new ArrayList<>();
7 namenListe.add("Alice");
8 namenListe.add("Bob");
9
10 // Konvertierung der List in ein Array für den API-Aufruf
11 String[] namenArray = namenListe.toArray(new String[0]);
12 sendeNamen(namenArray);
```

6 Statische Methoden von Arrays und Collections

In Java werden viele Operationen auf Arrays und Listen über statische Methoden in den Hilfsklassen `java.util.Arrays` und `java.util.Collections` ausgeführt.

In diesem Abschnitt geht es um:

- Sortieren von Arrays und Listen
- Suchen in sortierten Arrays/Listen mit `Arrays.binarySearch`
- Finden von Minimum und Maximum

6.1 Sortieren von Arrays mit `Arrays.sort`

Die Klasse `java.util.Arrays` stellt `sort`-Methoden für verschiedene Array-Typen bereit.

Primitive Arrays

```
1 int[] zahlen = {5, 3, 9, 1};
2
3 java.util.Arrays.sort(zahlen); // sortiert aufsteigend: 1, 3, 5, 9
```

Für primitive Typen wie `int`, `double`, `char` wird ein schneller, optimierter Sortieralgorithmus verwendet.

Objekt-Arrays (mit natürlicher Ordnung)

```
1 String[] namen = {"Bernd", "Anna", "Clara"};
2
3 java.util.Arrays.sort(namen); // nutzt die natürliche Ordnung von
    String
```

Hier wird die natürliche Ordnung verwendet, die durch das Interface `Comparable<T>` festgelegt ist (z.B. bei `String`, `Integer`,...).

Objekt-Arrays mit Comparator

```
1 import java.util.*;
2
3 String[] namen = {"Bernd", "Anna", "Clara"};
4
5 // Sortieren nach String-Länge
6 Arrays.sort(namen, Comparator.comparingInt(String::length));
```

Merkkasten

Merke: `Arrays.sort`

- Sortiert das übergebene Array *in-place* (Original wird verändert).
- Für Objekt-Arrays gilt: Entweder natürliche Ordnung (`Comparable`) oder eine explizit angegebene Ordnung (`Comparator`).
- Die Sortierung ist aufsteigend in Bezug auf die gewählte Ordnung.

6.2 Sortieren von Listen mit `Collections.sort` und `List.sort`

Zum Sortieren von Listen (z.B. `ArrayList`) wird die Klasse `java.util.Collections` verwendet – oder direkt die Methode `sort` am `List`-Interface.

Natürliche Ordnung

```
1  import java.util.*;
2
3  List<Integer> zahlen = new ArrayList<>();
4  Collections.addAll(zahlen, 5, 3, 9, 1);
5
6  Collections.sort(zahlen); // sortiert: [1, 3, 5, 9]
7  // alternativ ab Java 8:
8  zahlen.sort(null); // null -> natürliche Ordnung
```

Eigene Ordnung mit `Comparator`

```
1  List<String> namen = new ArrayList<>();
2  Collections.addAll(namen, "Bernd", "Anna", "Clara");
3
4  // Sortieren nach Länge des Namens (kürzeste zuerst)
5  Collections.sort(namen, Comparator.comparingInt(String::length));
6
7  // Das gleiche mit List.sort (empfohlene Schreibweise ab Java 8)
8  namen.sort(Comparator.comparingInt(String::length));
```

Merkkasten

Merke: `Collections.sort` vs. `List.sort`

- `Collections.sort(list)` ist der klassische Weg.
- `list.sort(comparator)` ist moderner und arbeitet direkt auf der Liste.
- Beide sortieren *in-place* und verändern die Reihenfolge der Elemente.

Stolperfallen

Stolperfallen beim Sortieren

- Unveränderliche Listen (z.B. aus `List.of(...)` oder manchen `Arrays.asList`-Varianten) können nicht sortiert werden (`UnsupportedOperationException`).
- Inkonsistente Implementierung von `compareTo` oder `Comparator` kann zu unerwarteten Sortierreihenfolgen führen.
- `null`-Werte in den Daten können zu `NullPointerException` führen, wenn nicht explizit behandelt (z.B. mit `Comparator.nullsFirst`).

6.3 Minimum und Maximum mit `Collections.min` und `Collections.max`

Für Collections (z.B. Listen) gibt es Hilfsmethoden, um das kleinste bzw. größte Element gemäß einer Ordnung zu ermitteln:

```
1 import java.util.*;
2
3 List<Integer> zahlen = Arrays.asList(5, 3, 9, 1);
4
5 int min = Collections.min(zahlen); // 1
6 int max = Collections.max(zahlen); // 9
```

Mit `Comparator` kann eine alternative Ordnung verwendet werden:

```
1 List<String> namen = Arrays.asList("Bernd", "Anna", "Clara");
2
3 // Längster Name
4 String laengsterName = Collections.max(
5     namen,
6     Comparator.comparingInt(String::length)
7 );
```

Merkkasten

Merke: Ordnung wiederverwendet

- Die gleiche Ordnung (natürlich oder per `Comparator`) wird verwendet für:
 - Sortieren (`sort`)
 - Suchen mit `binarySearch`
 - Finden von Minimum/Maximum (`min/max`)
- Für konsistente Ergebnisse sollten immer dieselben Vergleichsregeln benutzt werden.

7 Suchen mit Arrays und Collections

Nach dem Sortieren ist der nächste wichtige Schritt das **Suchen** von Elementen. Java bietet dafür sowohl lineare Suche (z.B. `contains`, `indexOf`) als auch binäre Suche über statische Methoden.

In diesem Abschnitt geht es um:

- Lineare Suche in Arrays und Listen
- Binäre Suche mit `Arrays.binarySearch` und `Collections.binarySearch`
- Interpretation der Rückgabewerte

7.1 Lineare Suche: contains und indexOf

Die einfachste Form der Suche ist die *lineare Suche*: Es wird von vorne nach hinten durch die Datenstruktur gelaufen, bis ein passendes Element gefunden wird.

Suche in Listen

```
1  import java.util.*;
2
3  List<String> namen = new ArrayList<>();
4  Collections.addAll(namen, "Anna", "Bernd", "Clara");
5
6  boolean hatAnna = namen.contains("Anna"); // true
7  int indexBernd = namen.indexOf("Bernd"); // 1
8  int indexDoris = namen.indexOf("Doris"); // -1 (nicht gefunden)
9
10 // Entfernen nach Wert (erstes Vorkommen)
11 boolean entfernt = namen.remove("Anna"); // true, falls vorhanden
```

Merkkasten

Merke: lineare Suche

- contains prüft nur, ob ein Element vorhanden ist.
- indexOf liefert den Index oder -1, wenn das Element nicht gefunden wurde.
- Aufwand ist im Worst-Case $O(n)$, da im Zweifel alle Elemente geprüft werden müssen.

Suche in Arrays Für Arrays gibt es keine Methoden contains oder indexOf direkt am Array. Man muss entweder selbst iterieren oder Hilfsmethoden nutzen (z.B. Streams oder eigene Schleifen).

```
1  int[] zahlen = {1, 4, 9, 16};
2
3  boolean hatNeun = false;
4  for (int z : zahlen) {
5      if (z == 9) {
6          hatNeun = true;
7          break;
8      }
9  }
```

7.2 Binäre Suche mit Arrays.binarySearch

Die binäre Suche ist deutlich schneller ($O(\log n)$), setzt aber voraus, dass das Array sortiert ist und dass dieselbe Ordnung verwendet wird wie beim Sortieren.

Primitive Arrays

```
1  int[] zahlen = {1, 3, 5, 7, 9};
2  java.util.Arrays.sort(zahlen); // sicherstellen, dass sortiert ist
3
4  int index5 = java.util.Arrays.binarySearch(zahlen, 5); // z.B. 2
5  int index4 = java.util.Arrays.binarySearch(zahlen, 4); // < 0 (nicht gefunden)
```

Objekt-Arrays mit natürlicher Ordnung

```
1 String[] namen = {"Bernd", "Anna", "Clara"};
2 java.util.Arrays.sort(namen); // ["Anna", "Bernd", "Clara"]
3
4 int idx = java.util.Arrays.binarySearch(namen, "Bernd"); // 1
```

Objekt-Arrays mit Comparator Wenn beim Sortieren ein Comparator verwendet wurde, muss `binarySearch` mit demselben Comparator aufgerufen werden.

```
1 import java.util.*;
2
3 String[] namen = {"Bernd", "Anna", "Clara"};
4 Comparator<String> cmp = Comparator.comparingInt(String::length);
5
6 Arrays.sort(namen, cmp);
7 int idx = Arrays.binarySearch(namen, "Anna", cmp);
```

Stolperfallen

Stolperfalle: binäre Suche

- **Nur für sortierte Arrays/Listen:** Wird auf unsortierten Daten aufgerufen, sind die Ergebnisse undefiniert.
- Die gleiche Ordnung muss für Sortierung und Suche verwendet werden (gleicher Comparator bzw. gleiche `compareTo`-Logik).
- Rückgabewerte < 0 bedeuten „nicht gefunden“ die genaue Bedeutung wird im nächsten Abschnitt erklärt.

7.3 Binäre Suche mit `Collections.binarySearch`

Für Listen bietet die Klasse `java.util.Collections` eine ähnliche Methode:

```
1 import java.util.*;
2
3 List<Integer> zahlen = new ArrayList<>();
4 Collections.addAll(zahlen, 1, 3, 5, 7, 9);
5 Collections.sort(zahlen);
6
7 int index5 = Collections.binarySearch(zahlen, 5); // z.B. 2
8 int index4 = Collections.binarySearch(zahlen, 4); // < 0 (nicht
    gefunden)
```

Mit Comparator funktioniert es analog:

```
1 List<String> namen = new ArrayList<>();
2 Collections.addAll(namen, "Bernd", "Anna", "Clara");
3
4 Comparator<String> cmp = Comparator.comparingInt(String::length);
5 Collections.sort(namen, cmp);
6
7 int idx = Collections.binarySearch(namen, "Anna", cmp);
```

7.4 Rückgabewert von `binarySearch` verstehen

`binarySearch` liefert:

- ≥ 0 : Index des gefundenen Elements.
- < 0 : Element wurde nicht gefunden.

Im Fall < 0 kann aus dem Rückgabewert die *Einfügeposition* (insertion point") berechnet werden:

Merkkasten

Formel für die Einfügeposition

Wenn das Ergebnis $r < 0$ ist, gilt:

$$insertionPoint = -r - 1$$

Das ist der Index, an dem das gesuchte Element eingefügt werden müsste, um die Sortierung beizubehalten.

```

1  int[] zahlen = {1, 3, 5, 7, 9};
2  int r = Arrays.binarySearch(zahlen, 4); // z.B. -3
3
4  if (r < 0) {
5      int insertionPoint = -r - 1; // ergibt 2
6      // 4 würde an Index 2 eingefügt, zwischen 3 und 5
7  }
```

Stolperfallen

Typische Fehler bei binarySearch

- Rückgabewert < 0 fälschlich als Indexinterpretieren.
- Aufruf mit anderem Comparator als beim Sortieren verwendet wurde.
- Vergessen, die Daten vorher zu sortieren.

8 Comparable und Comparator: natürliche und flexible Ordnung

Viele der zuvor gesehenen Methoden (z.B. `sort`, `binarySearch`, `min`, `max`) hängen davon ab, wie zwei Objekte miteinander verglichen werden können.

Dafür gibt es in Java zwei zentrale Konzepte:

- **Comparable<T>** – legt die *natürliche Ordnung* einer Klasse fest.
- **Comparator<T>** – erlaubt alternative, flexible Ordnungen von außen.

8.1 Natürliche Ordnung mit Comparable

Eine Klasse, die `Comparable<T>` implementiert, definiert ihre natürliche Reihenfolge über die Methode `compareTo`.


```

1 public class Person implements Comparable<Person> {
2     private String name;
3     private int alter;
4
5     public Person(String name, int alter) {
6         this.name = name;
7         this.alter = alter;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public int getAlter() {
15        return alter;
16    }
17
18    @Override
19    public int compareTo(Person other) {
20        // Natürliche Ordnung: nach Alter aufsteigend
21        return Integer.compare(this.alter, other.alter);
22    }
23 }

```

Damit können Arrays und Listen von `Person` direkt sortiert werden:

```

1 Person[] personenArray = {
2     new Person("Anna", 30),
3     new Person("Bernd", 25),
4     new Person("Clara", 40)
5 };
6
7 java.util.Arrays.sort(personenArray); // nutzt compareTo (Alter)

```

```

1 import java.util.*;
2
3 List<Person> personenListe = new ArrayList<>();
4 personenListe.add(new Person("Anna", 30));
5 personenListe.add(new Person("Bernd", 25));
6 personenListe.add(new Person("Clara", 40));
7
8 Collections.sort(personenListe); // ebenfalls nach Alter

```

Merkkasten

Merke: Comparable

- `compareTo` definiert die *natürliche Ordnung* einer Klasse.
- Wird verwendet von `Arrays.sort`, `Collections.sort`, `Collections.min`, `Collections.max`, `binarySearch` (ohne expliziten Comparator).
- Rückgabewert-Konvention:
 - `< 0`: `this < other`
 - `0`: `this` und `other` gelten als gleich
 - `> 0`: `this > other`

Stolperfallen

Stolperfällen bei compareTo

- **Inkonsistenz mit equals:** Wenn compareTo 0 liefert, sollten die Objekte inhaltlich "gleich" sein. Eine andere Logik bei equals kann zu merkwürdigem Verhalten in Sortierungen und Sets führen.
- **Keine Behandlung von null:** compareTo wird meist auf einem nicht-null Objekt aufgerufen; null führt leicht zu NullPointerException.
- **Komplexe Vergleichslogik:** Verschachtelte if/else-Konstrukte sind fehleranfällig – besser Hilfsmethoden wie Integer.compare verwenden.

8.2 Alternative Ordnungen mit Comparator

Ein Comparator<T> beschreibt eine Ordnung von außen – die Klasse selbst muss dafür nicht geändert werden. Praktisch, wenn:

- Sie eine andere Sortierung als die natürliche Ordnung brauchen.
- Sie eine fremde Klasse nicht ändern können oder dürfen.

Comparator als anonyme Klasse

```
1  import java.util.*;
2
3  List<Person> personen = new ArrayList<>();
4  personen.add(new Person("Anna", 30));
5  personen.add(new Person("Bernd", 25));
6  personen.add(new Person("Clara", 40));
7
8  // Sortieren nach Name (alphabetisch)
9  Collections.sort(personen, new Comparator<Person>() {
10     @Override
11     public int compare(Person p1, Person p2) {
12         return p1.getName().compareTo(p2.getName());
13     }
14 });
```

Comparator mit Lambda-Ausdruck Da Comparator<T> ein *funktionales Interface* ist (genau eine abstrakte Methode: compare), kann es sehr gut mit Lambdas verwendet werden:

```
1  // Sortieren nach Alter absteigend mit Lambda
2  personen.sort((p1, p2) -> Integer.compare(p2.getAlter(), p1.getAlter()));
3
4  // Sortieren nach Name, dann nach Alter (aufsteigend)
5  personen.sort(
6      Comparator.comparing(Person::getName)
7      .thenComparingInt(Person::getAlter)
8  );
```

8.3 Hilfsmethoden der Klasse Comparator

Die Klasse java.util.Comparator stellt viele statische Hilfsmethoden bereit, mit denen sich Comparatoren elegant bauen und kombinieren lassen.

```

1  import java.util.*;
2
3  List<String> namen = Arrays.asList("Bernd", "Anna", "Clara", "Alex");
4
5  // Nach Länge sortieren (kürzeste zuerst)
6  Comparator<String> nachLaenge = Comparator.comparingInt(String::length
7  );
8  namen.sort(nachLaenge);
9
10 // Nach Länge, dann alphabetisch
11 Comparator<String> nachLaengeDannName =
12 Comparator.comparingInt(String::length)
13 .thenComparing(Comparator.naturalOrder());
14
15 namen.sort(nachLaengeDannName);
16
17 // Längster Name mit Collections.max
String laengsterName = Collections.max(namen, nachLaenge);

```

Nützliche Hilfsmethoden (Auswahl):

- `Comparator.comparing(Function keyExtractor)` – erstellt Comparator nach einem Schlüssel.
- `Comparator.comparingInt/Long/Double(...)` – Variante für primitive Schlüssel.
- `thenComparing(...)` – Kettenvergleich (zweit-, dritt-Kriterium).
- `reversed()` – kehrt die Sortierreihenfolge um.
- `Comparator.nullsFirst(...)` / `nullsLast(...)` – behandelt null-Werte explizit.

Merkkasten

Comparable vs. Comparator – Kurzvergleich

- **Comparable:**
 - Ordnung ist in der Klasse selbst fest eingebaut.
 - Liefert eine natürliche Standard-Sortierung.
- **Comparator:**
 - Ordnung wird von außen vorgegeben.
 - Beliebig viele unterschiedliche Vergleichsstrategien möglich.
 - Ideal für Sortieren, Min/Max, `binarySearch` mit alternativen Kriterien.

8.4 Verbindung zu `sort`, `binarySearch`, `min` und `max`

Die bisher behandelten Methoden arbeiten je nach Überladung mit **natürlicher Ordnung (Comparable)** oder einem **übergebenen Comparator**:

- `Arrays.sort(T[] a)` – nutzt `compareTo` (Comparable).
- `Arrays.sort(T[] a, Comparator<? super T> c)` – nutzt den angegebenen Comparator.
- `Collections.sort(List<T> list)` – natürliche Ordnung (Comparable).
- `Collections.sort(List<T> list, Comparator<? super T> c)` – Comparator.

- `Collections.min/max(Collection<T> coll)` – natürliche Ordnung.
- `Collections.min/max(Collection<T> coll, Comparator<? super T> c)` – Comparator.
- `Arrays.binarySearch(...)` und `Collections.binarySearch(...)` – je nach Überladung mit oder ohne Comparator.

Stolperfallen

Konsistenz der Ordnung

- Sortieren und Suchen sollten immer mit *derselben* Ordnung arbeiten (gleiches `compareTo` bzw. gleicher `Comparator`).
- Werden verschiedene Vergleichsregeln gemischt, sind Ergebnisse von `binarySearch`, `min/max` und Sortierung nicht mehr verlässlich.

9 Funktionale Interfaces und Lambdas

Ab Java 8 hat Java Sprachmittel eingeführt, mit denen sich Funktionen als Werte verwenden lassen: **funktionale Interfaces** und **Lambda-Ausdrücke**.

Sie spielen eine zentrale Rolle bei:

- Comparatoren (Sortierung, Min/Max, binarySearch)
- dem Streams-API (map, filter, sorted,...)
- Callbacks (z.B. Listener in GUI-Frameworks)

9.1 Von anonymen Klassen zu Lambdas

Vor Java 8 wurden kurze Einmal-Implementierungen oft mit *anonymen Klassen* realisiert. Beispiel: ein Comparator als anonyme Klasse:

```
1  import java.util.*;
2
3  List<String> namen = new ArrayList<>();
4  Collections.addAll(namen, "Bernd", "Anna", "Clara");
5
6  // Anonyme Klasse als Comparator (vor Java 8 üblich)
7  Collections.sort(namen, new Comparator<String>() {
8      @Override
9      public int compare(String s1, String s2) {
10         return Integer.compare(s1.length(), s2.length());
11     }
12 });
13
14 tEM.out.println(namen);
```

Ab Java 8 kann derselbe Comparator als Lambda-Ausdruck geschrieben werden. Das ist möglich, weil Comparator<T> ein **funktionales Interface** ist.

```
1  // Das gleiche mit Lambda (ab Java 8)
2  Collections.sort(namen, (s1, s2) -> Integer.compare(s1.length(), s2.
3      length()));
4
5  // Oder noch kürzer mit Comparator-Hilfsmethode
6  namen.sort(Comparator.comparingInt(String::length));
```

Merkkasten

Von anonymer Klasse zu Lambda

- Anonyme Klasse: komplette Implementierung mit `new Interface() {...}`.
- Lambda: nur die *Implementierung der einen Methode* (Parameterliste und Rumpf).
- Voraussetzung: Das Interface muss genau **eine abstrakte Methode** haben $\Rightarrow$ funktionales Interface.

9.2 Was ist ein funktionales Interface?

Ein **funktionales Interface** ist ein Interface mit genau einer abstrakten Methode.

Typische Beispiele aus der Java-Standardbibliothek:

- Runnable – void run()

- `Comparator<T> – int compare(T a, T b)`
- `java.util.function.Predicate<T> – boolean test(T t)`
- `java.util.function.Function<T,R> – R apply(T t)`
- `java.util.function.Consumer<T> – void accept(T t)`
- `java.util.function.Supplier<T> – T get()`

Funktionale Interfaces werden häufig mit der Annotation `@FunctionalInterface` gekennzeichnet (optional, aber empfohlen):

```

1  @FunctionalInterface
2  public interface Rechner {
3      int berechne(int a, int b);
4  }

```

Stolperfallen

Funktional oder nicht?

- Ein funktionales Interface darf nur **eine** abstrakte Methode haben.
- Zusätzliche `default`- oder `static`-Methoden sind erlaubt.
- Die Annotation `@FunctionalInterface` ist optional, hilft aber, Fehler vom Compiler früh erkennen zu lassen.

9.3 Lambda-Grundsyntax

Eine Lambda-Funktion beschreibt kurz und prägnant, was eine Methode tun soll. Allgemeine Form:

`(parameter) -> { rumpf }`

Beispiele mit dem Interface `Rechner` von oben:

```

1  Rechner add = (a, b) -> a + b;
2  Rechner mul = (a, b) -> {
3      int produkt = a * b;
4      System.out.println("Produkt: " + produkt);
5      return produkt;
6  };
7
8  int summe = add.berechne(3, 4); // 7
9  int erg = mul.berechne(2, 5); // 10

```

Merkkasten

Lambda-Kurzregeln

- Parameter-Typen werden oft weggelassen: der Compiler leitet sie ab.
- Ein Ausdruck im Rumpf ohne geschweifte Klammern bedeutet: Rückgabewert dieses Ausdrucks.
- Mehrere Anweisungen brauchen `{... }` und ein explizites `return` bei Rückgabetypen.

9.4 Comparator als funktionales Interface

Im Zusammenhang mit Arrays und Collections sind Lambdas vor allem bei `Comparator<T>` spannend.

Beispiele mit Comparator und Lambdas

```
1  import java.util.*;
2
3  List<String> namen = Arrays.asList("Bernd", "Anna", "Clara", "Alex");
4
5  // Alphabetisch absteigend
6  namen.sort((s1, s2) -> s2.compareTo(s1));
7
8  // Nach Länge absteigend
9  namen.sort((s1, s2) -> Integer.compare(s2.length(), s1.length()));
10
11 // Mit Hilfsmethoden: nach Länge, dann alphabetisch
12 namen.sort(
13     Comparator.comparingInt(String::length)
14     .thenComparing(Comparator.naturalOrder())
15 );
```

Lambdas können auch zusammen mit `Collections.max` und `Collections.min` verwendet werden:

```
1  List<String> namen2 = Arrays.asList("Tom", "Christopher", "Eve");
2
3  String laengsterName = Collections.max(
4      namen2,
5      Comparator.comparingInt(String::length)
6  );
```

9.5 Funktionale Interfaces im Zusammenspiel mit Streams

Das Streams-API (ab Java 8) nutzt funktionale Interfaces intensiv. Beispiele:

- `filter` erwartet ein `Predicate<T>`
- `map` erwartet eine `Function<T,R>`
- `forEach` erwartet einen `Consumer<T>`
- `sorted` erwartet optional einen `Comparator<T>`

Beispiel: Filtern, Sortieren und Ausgeben mit Streams

```
1  import java.util.*;
2
3  List<String> namen = Arrays.asList("Anna", "Bernd", "Alex", "Clara");
4
5  namen.stream()
6      .filter(s -> s.startsWith("A")) // Predicate<String>
7      .sorted(Comparator.comparingInt(String::length)) // Comparator<String>
8      .forEach(s -> System.out.println(s)); // Consumer<String>
```

Merkkasten

Streams + Lambdas: Verbindung zu Arrays und Collections

- Ausgangspunkt sind oft `List<T>` (z.B. `ArrayList`) oder Arrays.
- Sortierlogik, die Sie mit `Comparator` für `Collections.sort` definiert haben, lässt sich meist direkt in Streams mit `sorted(cmp)` wiederverwenden.
- `list.stream()` nutzt dieselben Vergleichsregeln wie `Collections.sort` und `Collections.max`, wenn dieselben Comparatoren verwendet werden.

9.6 Zusammenfassung: Warum das für Arrays und ArrayList wichtig ist

- **Comparable** und **Comparator** bestimmen die Ordnung, die von `Arrays.sort`, `Collections.sort`, `binarySearch`, `min` und `max` genutzt wird.
- **Comparator** ist ein funktionales Interface und damit ideal für Lambda-Ausdrücke.
- Mit Lambdas wird Code für Vergleiche deutlich kürzer und besser lesbar als mit anonymen Klassen.
- Das Streams-API baut auf denselben funktionalen Interfaces auf und erlaubt es, Arrays und Listen noch komfortabler zu verarbeiten.

10 Zusammenfassung und Spickzettel

Dieser Abschnitt fasst die wichtigsten Punkte rund um Arrays, `ArrayList`, Collections, Vergleichen, Suchen und Lambdas kurz zusammen.

10.1 Arrays vs. ArrayList – Schnellentscheidung

Frage	Eher Array	Eher ArrayList
Größe vorher bekannt?	Ja, feste Anzahl von Elementen	Nein, Anzahl ändert sich zur Laufzeit
Viele Primitive (int, double,...)?	Ja, wegen Performance/kein Autoboxing	Nur wenn Komfort wichtiger ist als Performance
Häufiges Hinzufügen/Entfernen?	Aufwendig (kopieren, neues Array)	Einfach mit <code>add/remove</code>
Nutzung der Collections-API/Streams?	Nur über Umwege	Natürliche Wahl (<code>List<T></code>)
Speicher stark begrenzt?	Besser geeignet	Mehr Overhead durch Objekte

10.2 Wichtige Klassen und ihr Zweck

- `java.util.Arrays` – statische Methoden für Arrays
 - `sort(array)` – Array sortieren (natürliche Ordnung oder mit `Comparator`)
 - `binarySearch(array, key)` – binäre Suche im sortierten Array
 - `asList(array)` – Array als feste List-Sicht
- `java.util.Collections` – statische Methoden für Collections

- `sort(list)` / `list.sort(cmp)` – Liste sortieren
- `binarySearch(list, key)` – binäre Suche in sortierter Liste
- `min(coll)`, `max(coll)` – kleinstes/größtes Element
- `synchronizedList(list)` – einfache Synchronisierung
- `java.util.Comparator` – flexible Vergleichslogik
 - `comparing(...)` / `comparingInt(...)` – Comparator aus Schlüssel bauen
 - `thenComparing(...)` – Zweit-/Drittsortierkriterium
 - `reversed()` – Reihenfolge umdrehen

10.3 Vergleichen und Sortieren auf einen Blick

- **Natürliche Ordnung** – Klasse implementiert `Comparable<T>`
 - Methoden nutzen automatisch `compareTo`: `Arrays.sort`, `Collections.sort`, `min/max`, `binarySearch` (ohne Comparator).
- **Alternative Ordnung** – `Comparator<T>`
 - Wird als zusätzliches Argument übergeben: `sort(..., cmp)`, `max(..., cmp)`, `binarySearch(..., cmp)`.
 - Lässt sich bequem mit Lambda schreiben: `(a, b) -> ...`
- **In-place-Sortierung**
 - `Arrays.sort` und `Collections.sort` verändern das Original.
 - Für unveränderliche Listen zuerst eine kopierbare Liste erstellen (z.B. `new ArrayList<>(...)`).

10.4 Suchen: linear vs. binär

- **Lineare Suche**
 - Listen: `contains`, `indexOf` ($O(n)$).
 - Arrays: eigene Schleife oder Streams.
- **Binäre Suche mit `binarySearch`** ($O(\log n)$)
 - Voraussetzung: Daten sind gemäß derselben Ordnung sortiert.
 - Rückgabewert r :
 - * $r \geq 0$: Element an Index r gefunden.
 - * $r < 0$: nicht gefunden, Einfügeposition: `insertionPoint = -r - 1`.

10.5 Konvertierung: Arrays $\leftrightarrow$ List

- **Array $\rightarrow$ List**
 - `List<T> list = Arrays.asList(array)`; – feste Größe, `set` erlaubt, `add/remove` nicht.
 - `List<T> mutable = new ArrayList<>(Arrays.asList(array))`; – wirklich dynamische Liste.
- **List $\rightarrow$ Array**
 - `T[] arr = list.toArray(new T[0]);`

10.6 Funktionale Interfaces und Lambdas – Kurzüberblick

- **Funktionales Interface:** genau eine abstrakte Methode.
 - Beispiele: `Runnable`, `Comparator<T>`, `Predicate<T>`, `Function<T,R>`.
- **Lambda-Grundform:** `(parameter) -> rumpf`
 - Kurzformen ohne Typen und ohne Klammern bei einem Parameter möglich.
 - Bei einem Ausdruck kann `return` und die geschweifte Klammer entfallen.
- **Comparator mit Lambda**
 - `list.sort((a, b) -> a.compareTo(b));`
 - `list.sort(Comparator.comparingInt(String::length));`
- **Streams + Lambdas**
 - `filter (Predicate), map (Function), forEach (Consumer), sorted (Comparator)`.
 - Beispiel: `list.stream().filter(p -> p.getAlter() > 18).sorted(cmp).forEach(...);`

10.7 Minimal-Spickzettel: „Wenn du X brauchst, nimm Y“

- **Feste Anzahl von ints, maximale Performance** → `Array` (z.B. `int []` mit `Arrays.sort`).
- **Dynamische Liste von Objekten** → `ArrayList<T>` + `Collections.sort` / `list.sort`.
- **Einmalige Sortierung in natürlicher Reihenfolge** → `Comparable` implementieren und `sort(list)` oder `Arrays.sort(array)`.
- **Unterschiedliche Sortierreihenfolgen** → `Comparator` mit Lambdas.
- **Schnelle Suche in sortierten Daten** → `binarySearch` (`Arrays/Collections`).
- **Längstes/kürzestes Element** → `Collections.max/min` mit passendem `Comparator`.
- **Komplexe Verarbeitungsketten** → `Streams` (`list.stream()`...).

Merkkasten

Take-away

Wenn Sie wissen, wie `Arrays`, `ArrayList`, `Arrays.*`, `Collections.*`, `Comparable`, `Comparator` und Lambdas zusammenspielen, haben Sie einen Großteil des Alltags mit Sammlungen in Java im Griff.

A Cheat Sheet: Arrays, ArrayList, Collections, Lambdas

Arrays vs. ArrayList

Frage	Array	ArrayList
Größe	fest	dynamisch
Typen	primitive + Objekte	nur Objekte (Wrapper)
API	wenig (<code>Arrays.*</code>)	viele Methoden (<code>List</code>)
Performance	sehr gut für Primitive	etwas Overhead

Wichtige Klassen

- **Arrays** – statische Methoden für Arrays
- **Collections** – statische Methoden für Collections
- **List / ArrayList** – dynamische Listen
- **Comparable** – natürliche Ordnung (`compareTo`)
- **Comparator** – alternative Ordnung (`compare`)

Arrays – wichtigste Methoden

```
Arrays.sort(array) // Array sortieren
Arrays.sort(array, comparator) // mit Comparator
Arrays.binarySearch(array, key) // binäre Suche
Arrays.binarySearch(array, key, cmp)
Arrays.asList(array) // Array -> feste List-Sicht
```

Collections – wichtigste Methoden

```
Collections.sort(list) // natürliche Ordnung
Collections.sort(list, cmp) // mit Comparator
list.sort(cmp) // ab Java 8, bevorzugt
```

```
Collections.binarySearch(list, key)
Collections.binarySearch(list, key, cmp)
```

```
Collections.min(coll) // kleinstes Element
Collections.max(coll) // größtes Element
Collections.min(coll, cmp)
Collections.max(coll, cmp)
```

```
Collections.synchronizedList(list) // einfache Synchronisierung
```

Konvertierung Arrays ↔ List

```
// Array -> List (feste Größe)
List<T> l = Arrays.asList(array);

// Array -> wirklich dynamische List
List<T> l = new ArrayList<>(Arrays.asList(array));

// List -> Array
T[] arr = list.toArray(new T[0]);
```

Comparable vs. Comparator

```
class Person implements Comparable<Person> {
    public int compareTo(Person o) {... }
}

// nutzt Comparable
Arrays.sort(personen);
Collections.sort(personenListe);

// Comparator (Lambda)
Comparator<Person> cmp =
    (p1, p2) -> Integer.compare(p1.getAlter(), p2.getAlter());

Collections.sort(personenListe, cmp);
personenListe.sort(cmp);
```

Comparator-Hilfsmethoden

```
Comparator.comparing(Person::getName)
Comparator.comparingInt(Person::getAlter)

Comparator.comparingInt(String::length)
    .thenComparing(Comparator.naturalOrder())

cmp.reversed() // umgekehrte Reihenfolge
Comparator.nullsFirst(cmp)
Comparator.nullsLast(cmp)
```

Lambdas und Streams – Minimalbeispiele

```
// Lambda-Grundform
(param) -> ausdruck
(a, b) -> {...; return wert; }

// Comparator mit Lambda
list.sort((a, b) -> a.compareTo(b));
list.sort(Comparator.comparingInt(String::length));

// Streams
list.stream()
    .filter(x -> x > 0)
    .sorted()
    .forEach(x -> System.out.println(x));
```

B Prüfungs-Checkliste

Dieser Abschnitt listet die wichtigsten Punkte auf, die Sie vor einer Prüfung zu Arrays, ArrayList, Collections und Lambdas beherrschen sollten.

A. Arrays vs. ArrayList

- Unterschied feste vs. dynamische Größe erklären können.
- Wissen, dass Arrays Primitive direkt speichern, ArrayList nur Objekte.

- Typische Einsatzzwecke für Array vs. ArrayList nennen können.

B. Syntax und Basis-Operationen

Arrays

- Deklaration und Initialisierung: `int[] a = new int[5];`, `int[] a = {1,2,3};`.
- Zugriff mit Index und Schleifen (`for`, `for-each`).

ArrayList

- Deklaration: `List<T> l = new ArrayList<>();`.
- Wichtige Methoden: `add`, `get`, `set`, `remove`, `size`, `contains`, `indexOf`.

C. Konvertierung Arrays ↔ List

- Array → List: `Arrays.asList(array)` (feste Größe!).
- Mutable List: `new ArrayList<>(Arrays.asList(array))`.
- List → Array: `T[] arr = list.toArray(new T[0]);`.
- Stolperfalle: `Arrays.asList` erlaubt kein `add/remove`.

D. Statische Methoden: Arrays und Collections

Arrays

- `Arrays.sort(array)` und `Arrays.sort(array, cmp)`.
- `Arrays.binarySearch(array, key[, cmp])`.
- `Arrays.asList(array)`.

Collections

- `Collections.sort(list)` und `list.sort(cmp)`.
- `Collections.binarySearch(list, key[, cmp])`.
- `Collections.min/max(coll[, cmp])`.

E. Suchen: linear vs. binär

- Lineare Suche in Listen: `contains`, `indexOf` ($O(n)$).
- Binäre Suche mit `binarySearch` ($O(\log n)$) nur auf sortierten Daten.
- Rückgabewert: $r \geq 0$ Index, $r < 0$ nicht gefunden, `insertionPoint` = $-r - 1$.

F. Comparable vs. Comparator

- `Comparable<T>`: natürliche Ordnung über `compareTo`.
- `Comparator<T>`: alternative Ordnung von außen über `compare`.
- Wissen, welche `sort/min/max/binarySearch`-Überladungen welchen Mechanismus nutzen.

G. Funktionale Interfaces und Lambdas

- Begriff *funktionales Interface* kennen (eine abstrakte Methode).
- Beispiele: `Comparator`, `Predicate`, `Function`, `Consumer`.
- Lambda-Grundsyntax: `(param) -> ausdruck`.
- Einen einfachen `Comparator` als Lambda schreiben können.
- Grundidee von Streams: `list.stream().filter(...).sorted(...).forEach(...);`.

H. Typische Stolperfallen

- `ArrayIndexOutOfBoundsException` bei falschen Indizes.
- Autoboxing-Overhead bei großen `ArrayList<Integer>` etc.
- `Arrays.asList` liefert Liste fester Größe.
- `binarySearch` auf unsortierten Daten.
- Inkonsistente Implementierung von `compareTo` und `equals`.